Performance Analysis of TCP and UDP Under Emulated Network Conditions Using Socket Programming and Wireshark

Course: Computer Networks | **Team size:** 1 | **Platform:** Linux (Kubuntu) | **Language:** Python 3 **Tools:** Python `socket`, Linux network namespaces, `tc netem`, Wireshark/ `tshark`, Matplotlib

> **Status note:** This is the project plan and report skeleton. Every number, graph and observation in Section 7 must come from your own runs. Nothing here is a pre-written result.

Abstract

TCP and UDP are the two main transport protocols on IP networks. TCP gives a reliable, ordered byte stream using sequence numbers, acknowledgements, retransmission and congestion control [1][3]. UDP gives a minimal datagram service in which delivery and duplicate protection are not guaranteed [2]. This project measures what those design differences cost and buy in practice.

I implement TCP and UDP client-server programs in Python and run them across a three-node virtual topology (client, router, server) built with Linux network namespaces. The Linux `netem` emulator adds controlled delay, packet loss and a bandwidth bottleneck at the router [4]. Transfer time, goodput, UDP delivery ratio and TCP retransmissions are measured over sweeps of loss and delay, with repeated trials. Wireshark/ `tshark` provide packet-level evidence (handshake, ACKs, retransmissions, UDP datagrams) [5]. The measured curves are then explained with transport-layer theory, including the loss/RTT dependence of TCP throughput [7].

The project does not claim a new protocol or that one protocol is better. It builds a reproducible experiment that connects protocol theory to measured behaviour.

---

1. Introduction

1.1 Background

At the transport layer, TCP provides connection setup (SYN, SYN-ACK, ACK), sequence-number-based reliability, retransmission, flow control and congestion control [1][3]. UDP has no connection setup and no delivery guarantee; applications that need reliability must add it themselves [2]. These differences are usually taught in theory. Here they are observed in real traffic under controlled conditions.

1.2 Motivation

A basic TCP/UDP socket exercise shows that data moves, but says nothing quantitative about behaviour under impairment. This project follows the chain:

theory → socket implementation → emulated impairment → packet capture → measurement → explanation

Published comparisons of TCP and UDP already exist, mostly in simulators such as NS-3 [8][9]. This project uses real sockets, the real Linux TCP stack and real packet captures on a small emulated network, which is feasible for one student.

2. Problem Statement and Objectives

Problem statement: How do TCP and UDP differ in measured performance and packet-level behaviour when the same transfer is subjected to controlled delay, packet loss and a bandwidth bottleneck?

Objectives

1. Implement TCP and UDP client-server transfer programs in Python.
2. Build a reproducible virtual topology with controllable delay, loss and bandwidth.
3. Measure transfer time, goodput, UDP delivery ratio and TCP retransmissions.
4. Sweep loss and delay to obtain trends, not isolated points.
5. Capture traffic and identify handshake, ACK, retransmission and datagram behaviour in Wireshark.
6. Explain the results using TCP/UDP mechanisms and a simple analytical model.
7. State clearly the limits of an emulated environment.

Guiding questions (kept short, not a formal "hypothesis" section)

- Q1: How do the two protocols compare with no impairment?
- Q2: How does added RTT affect each?
- Q3: How does packet loss affect TCP goodput and retransmissions, and UDP delivery?
- Q4: Does packet-level evidence explain the measured differences?

---

3. Background

3.1 TCP

TCP is connection-oriented and provides a reliable, in-order byte stream (RFC 9293, August 2022, which obsoletes RFC 793) [1]. Loss is detected through sequence numbers and repaired by retransmission. Congestion control (slow start, congestion avoidance, fast retransmit, fast recovery) is specified in RFC 5681 [3]. Consequence for this project: when loss or delay changes, TCP changes its sending rate, so throughput reflects the network condition.

3.2 UDP

UDP (RFC 768) is a datagram service with minimal mechanism; delivery and duplicate protection are not guaranteed [2]. It does not slow down on loss, so any loss is visible to the application unless the application repairs it.

3.3 Linux netem

`netem` is a queueing discipline for emulating network properties. It supports delay (with jitter), random and burst loss, duplication, corruption, reordering and rate limiting [4]. Its manual page lists two limitations that matter here:

- It is limited by kernel timer granularity, so delay and rate may be affected by clock interrupts.
- Because of TCP Small Queues (TSQ), for TCP results to be realistic netem must be placed on the

Because of TCP Small Queues (TSQ), for TCP results to be realistic netem must be placed on the **ingress of the receiver host** [4].

This second point shapes the topology in Section 5.1: putting netem on a separate router node, rather than on the sending host's own egress, avoids the TSQ interaction.

3.4 Wireshark TCP analysis

Wireshark tracks TCP state and flags conditions such as *TCP Retransmission*, *TCP Fast Retransmission*, *TCP Spurious Retransmission*, *TCP Dup ACK*, *Out-Of-Order* and *Previous segment not captured* [5]. Retransmission flags are derived from sequence and acknowledgement numbers and depend on where the capture was taken. Therefore captures should be taken at the sender's interface and retransmission counting should be documented.

---

4. Related Work

- Guo et al. (2025) compare TCP and UDP in NS-3 using throughput, end-to-end delay, lost packets, packet-loss ratio, bandwidth utilisation, jitter and packet delivery ratio, and report that TCP performs well but is affected by bandwidth and delay [8].
- AL-Dhief et al. (2018) compare TCP and UDP across simulation scenarios [9].
- A university networking lab (Duke COMPSCI 356) uses `netem` on the loopback interface to test a reliable transport implemented over UDP [10]. This shows that netem-based transport experiments, and building reliability on UDP, are standard coursework practice.

Positioning: those simulator studies model the stack; this project measures the real Linux stack and real captures. That is a difference of method, not a claim of novelty.

---

5. Methodology

5.1 Testbed topology

Three network namespaces connected by two virtual Ethernet (veth) pairs:

```
[client ns] ---- veth ---- [router ns] ---- veth ---- [server ns]
 10.0.1.1                10.0.1.2 | 10.0.2.2              10.0.2.1
                         (ip_forward=1, netem here)
```

Design decisions:

- **netem on the router**, not on the endpoints. This gives a real forwarding hop and avoids the TSQ issue in [4].
- **Delay applied on both router interfaces** so that added round-trip time equals twice the per-direction value. Example: 50 ms per direction gives about 100 ms added RTT. The report states RTT, not per-direction delay.
- **Loss applied only in the client→server direction** (router interface facing the server). Configured loss then equals data-packet loss, and ACK loss is not mixed in. State this in the report.
- **Bandwidth bottleneck** (for example 10 Mbit/s via `netem rate`) on the router. Without it, veth speed

is effectively very high, transfers finish in milliseconds and results become dominated by Python and kernel overhead.

- **Offloads disabled** on the veth interfaces (`ethtool -K <if> tso off gso off gro off`), otherwise captures show oversized frames and packet counts stop matching the network model.

Fallback: if namespaces cause trouble, run on the loopback interface with `netem` as done in [10]. On loopback, delay applies in both directions (so RTT = 2 × delay) and loss also hits ACKs. Document this if used.

5.2 Implementation

TCP (client/server)

- Client connects, sends *N* bytes in fixed-size chunks, then waits for a 1-byte application acknowledgement from the server.
- Server records time of first byte received, time of last byte received, and byte count, then sends the 1-byte acknowledgement.

UDP (client/server)

- Client splits the payload into datagrams of fixed size (for example 1200 bytes of payload) with a header carrying a 32-bit sequence number.
- Sender is **paced** to a fixed rate below the bottleneck, so loss observed at the receiver comes from netem and not from sender bursts or receive-buffer overflow.
- Server enlarges `SO_RCVBUF`, records every sequence number, and computes missing, duplicate and out-of-order counts.
- An end-of-transfer marker is sent several times, because a single marker can itself be lost. The server also uses an idle timeout.
- No retransmission in the base version.

Measurement driver

- One script loops over condition × protocol × trial, applies netem, starts the capture, runs server and client, stores a CSV row, stops the capture, and resets netem.
- `tshark` counts retransmissions automatically from the capture; use a filter such as `tcp.analysis.retransmission || tcp.analysis.fast_retransmission` and record exactly which flags were counted [5].
- Cross-check with the kernel's own counters (for example `nstat` or `ss -ti` inside the client namespace).

5.3 Experimental design

Set	Variable swept	Fixed
E1 Baseline	none	0 added delay, 0 % loss, 10 Mbit/s
E2 Delay	added RTT: 0, 50, 100, 200 ms	0 % loss
E3 Loss	data-direction loss: 0, 0.5, 1, 2, 5	fixed small delay (for example 20 ms

	%	RTT)
E4 Size (optional)	payload: 1, 10 MB	baseline
E5 Reliable UDP (stretch)	as E3	see 5.6

- Trials: 5 per point (more if variance is high). Report mean and standard deviation.
- Same machine, payload, and measurement method for both protocols.
- Final values (payload size, pacing rate, rates) are set after a pilot run and then frozen.

5.4 Metrics

Transfer time: T = t_last_byte − t_first_byte (measured at the server; TCP client-side time additionally ends at the application acknowledgement).

Goodput (application level):

Goodput = (8 × bytes_delivered_to_application) / T (bit/s)

For UDP, bytes_delivered counts only datagrams that arrived. Report clearly that this is not link throughput.

UDP delivery / loss:

Loss % = (packets_sent − packets_received) / packets_sent × 100 Delivery ratio = packets_received / packets_sent × 100

Also record duplicates and out-of-order arrivals.

TCP retransmissions: count from `tshark` (and the kernel counter as a cross-check).

Sanity check: observed UDP loss should track configured loss. Give a confidence note: with about 8,000 datagrams and 1 % loss, the expected number of losses is about 80, so run-to-run variation is small. With 700 datagrams (about 1 MB) it is only about 7, so results are noisy. This is why 10 MB payloads are recommended for the loss sweep.

5.5 Analytical reference

Two simple relations are used to interpret results (not to replace measurement):

- **Bandwidth-delay product:** BDP = bandwidth × RTT. A TCP flow needs a window of roughly BDP to fill the link, so at high RTT and short transfers, slow start dominates.
- **Loss-limited TCP throughput (Mathis et al.):** BW ≈ (MSS / RTT) × (C / √p), with C ≈ 1.22 for periodic loss and Reno-style behaviour [7]. Linux uses CUBIC by default, so treat this as a trend guide (throughput falls roughly with 1/√p and 1/RTT). Optionally set Reno per socket (`TCP_CONGESTION`) to compare closely with the model.

5.6 Optional stretch: reliable UDP

If the core is complete, add a stop-and-wait or sliding-window acknowledgement scheme over UDP and compare against TCP under the E3 loss sweep. This turns the study into "what does TCP's reliability cost, and can it be rebuilt?", and mirrors practice in coursework such as [10]. Cut this first if time runs short.

5.7 Wireshark analysis plan

- **TCP handshake:** filter `tcp.flags.syn==1`; screenshot SYN, SYN-ACK, ACK.
- **Data and ACKs:** `tcp.port==<PORT>`.
- **Loss run:** filter `tcp.analysis.flags`; show retransmissions and duplicate ACKs, and the time gap before a retransmission.
- **UDP:** `udp.port==<PORT>`; show that there is no handshake and that datagram sequence gaps match the receiver's log.
- Capture at the client interface (sender side). Save every capture as `.pcapng` with a descriptive name.

---

6. Expected Behaviour (Hypotheses, Not Results)

These follow from theory and are to be confirmed or refuted by data:

- Adding RTT should increase TCP transfer time, especially for short transfers, because of slow start and the window/RTT relationship. A paced UDP sender should be largely insensitive to RTT apart from a one-time added latency.
- As loss increases, TCP goodput should fall, and retransmission counts should rise. UDP goodput should fall roughly in proportion to the loss.
- UDP loss should be visible as missing sequence numbers matching the configured loss rate, provided pacing and buffers are set correctly.

If measurements disagree, that is a finding. Explain it (buffering, timer granularity, CPU load, offloads, congestion-control variant).

---

7. Results (to be completed from experiments)

7.1 Baseline (E1)

Metric	TCP	UDP
Mean transfer time (s)		
Mean goodput (Mbit/s)		
Retransmissions		n/a
Observed loss (%)	n/a	

7.2 Delay sweep (E2)

Added RTT	TCP time	TCP goodput	UDP time	UDP goodput
0 / 50 / 100 / 200 ms				

7.3 Loss sweep (E3)

7.3 Loss sweep (E3)

Configured loss	TCP goodput	TCP retransmissions	UDP observed loss	UDP goodput
0 / 0.5 / 1 / 2 / 5 %				

7.4 Graphs

1. Baseline goodput, TCP vs UDP.
2. Transfer time vs added RTT (both protocols).
3. Goodput vs loss (both protocols, with error bars; optionally overlay the Mathis curve).
4. TCP retransmissions vs loss.
5. UDP observed loss vs configured loss (should track the diagonal).

7.5 Packet-level evidence

Handshake screenshot, retransmission screenshot from the loss run, UDP capture with a sequence gap, each with a caption explaining what it shows.

---

8. Discussion Framework

For each experiment answer: **What was measured? Why (which mechanism)? Was it consistent across trials? Does it answer the guiding question? What limits the conclusion?** Do not restate table values; explain them.

---

9. Limitations

- Emulated, single-machine environment; namespaces share one kernel, so CPU scheduling, Python overhead and timer granularity affect timing [4].
- Random (independent) loss is simpler than real loss, which often comes in bursts. `netem` also supports burst-loss models [4] (future work).
- Loss is applied to the data direction only (by design), so ACK loss effects are not studied.
- CUBIC (Linux default) differs from the Reno model behind the analytical formula [7].
- Goodput is an application-level measure, not link capacity.
- Results apply to this setup and must not be generalised to the Internet.

---

10. Timeline (14 days)

Days	Work
1–2	TCP and UDP transfers working; namespace topology up; netem verified with `ping` and a quick transfer
3–4	Timing, CSV logging, paced UDP with sequence numbers, receiver-side loss log

5–6	Automated driver; `tshark` retransmission counting; pilot runs; freeze parameters
7–9	Full sweeps E1–E3 with saved captures
10	Graphs; investigate anomalies
11–12	Write report (about 10–12 pages)
13	Slides; demo rehearsal; recorded backup
14	Buffer (E4 or reliable UDP only if everything else is done)

11. Deliverables and Repository Layout

```
project/
├── client/   tcp_client.py  udp_client.py
├── server/   tcp_server.py  udp_server.py
├── setup/    topology.sh    netem.sh   reset.sh
├── experiments/ run_experiment.py
├── results/  raw_results.csv  processed_results.csv
├── analysis/ graphs.py
├── captures/ *.pcapng
└── README.md   (requirements, commands, parameters, how to reproduce)
```

Deliverables: source code, raw and processed CSV, captures, graphs, this report, and a 10–12 slide presentation with a live demo and a recorded backup.

Appendix A. Topology and netem sketch (verify on your machine)

```
# namespaces and links
sudo ip netns add client; sudo ip netns add router; sudo ip netns add server
sudo ip link add c-r type veth peer name r-c
sudo ip link add s-r type veth peer name r-s
sudo ip link set c-r netns client;  sudo ip link set r-c netns router
sudo ip link set s-r netns server;  sudo ip link set r-s netns router

# addresses
sudo ip -n client addr add 10.0.1.1/24 dev c-r
sudo ip -n router addr add 10.0.1.2/24 dev r-c
sudo ip -n router addr add 10.0.2.2/24 dev r-s
sudo ip -n server addr add 10.0.2.1/24 dev s-r

# up + routes + forwarding
for n in client router server; do sudo ip -n $n link set lo up; done
sudo ip -n client link set c-r up; sudo ip -n router link set r-c up
sudo ip -n router link set r-s up; sudo ip -n server link set s-r up
sudo ip -n client route add default via 10.0.1.2
sudo ip -n server route add default via 10.0.2.2
sudo ip netns exec router sysctl -w net.ipv4.ip_forward=1
```

```
# example impairment: 50 ms each way (about 100 ms RTT), 1 % loss client->server, 10 Mbit
sudo ip netns exec router tc qdisc add dev r-s root netem delay 50ms loss 1% rate 10mbit
sudo ip netns exec router tc qdisc add dev r-c root netem delay 50ms rate 10mbit

# reset
sudo ip netns exec router tc qdisc del dev r-s root
sudo ip netns exec router tc qdisc del dev r-c root
```

Before trusting any run, verify with `ping` (RTT matches configuration) and a short transfer. Very high delay with `rate` may need a larger `limit`; check `tc -s qdisc` for drops caused by the queue rather than by configured loss.

References

[1] W. Eddy, Ed., "Transmission Control Protocol (TCP)," RFC 9293, IETF, Aug. 2022. https://www.rfc-editor.org/rfc/rfc9293.html

[2] J. Postel, "User Datagram Protocol," RFC 768, IETF, Aug. 1980. https://www.rfc-editor.org/rfc/rfc768.html

[3] M. Allman, V. Paxson, E. Blanton, "TCP Congestion Control," RFC 5681, IETF, Sep. 2009. https://www.rfc-editor.org/rfc/rfc5681.html

[4] `tc-netem(8)`, Linux manual page (iproute2). https://man7.org/linux/man-pages/man8/tc-netem.8.html

[5] Wireshark User's Guide, "7.5 TCP Analysis." https://www.wireshark.org/docs/wsug_html_chunked/ChAdvTCPAnalysis.html

[6] Python Software Foundation, "Socket Programming HOWTO." https://docs.python.org/3/howto/sockets.html

[7] M. Mathis, J. Semke, J. Mahdavi, T. Ott, "The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm," ACM SIGCOMM Computer Communication Review, vol. 27, no. 3, 1997.

[8] Guo Gaoyang, Cui Zhiyi, Faizan Qamar, Azana Hafizah Mohd Aman, "A Comparative Study of TCP and UDP Performance Using NS-3 Simulation," Asia-Pacific Journal of Information Technology and Multimedia, vol. 14, no. 1, 2025. DOI: 10.17576/apjitm-2025-1401-01

[9] F. AL-Dhief et al., "Performance Comparison between TCP and UDP Protocols in Different Simulation Scenarios," International Journal of Engineering & Technology, vol. 7, no. 4.36, pp. 172–176, 2018.

[10] Duke University, COMPSCI 356, "Lab 1: Reliable Transport" (Spring 2014). https://courses.cs.duke.edu/compsci356/spring14/labs/lab1/lab1.htm